

Algoritmos de optimización - Seminario.

Nombre y Apellidos: Adeldo Steve Araya Solórzano

Url: <https://github.com/SteveAraya/viu-algoritmos>

Problema: 1. Sesiones de doblaje

Descripción del problema.

Se precisa coordinar el doblaje de una película. Los actores de doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por **cada día** que deben desplazarse hasta el estudio de grabación **independientemente del número de tomas** que se graben. No es posible grabar más de **6 tomas por día**.

El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible.

Los datos son:

- Número de actores: **10**
- Número de tomas: **30**
- Matriz Actores/Tomas: **1** indica que el actor participa en la toma, **0** en caso contrario.

Nota sobre la unidad de coste. Como todos los actores cobran lo mismo por día, el gasto total es proporcional al número de pares (actor, día) en los que el actor acude al estudio. A esa magnitud la llamamos **días-actor**, y es lo que minimizamos.

(*) La respuesta es obligatoria

0. Datos del problema y modelo de representación.

La matriz se incrusta directamente en el notebook para que sea reproducible sin depender de ficheros externos (por ejemplo, al ejecutarlo en Google Colab).

```
In [1]: # -----  
#  
# Matriz Actores/Tomas del enunciado: 30 tomas x 10 actores.  
# Cada fila es una toma; cada columna, un actor.  
# -----  
MATRIZ = [
```

```

[1, 1, 1, 1, 1, 0, 0, 0, 0, 0], # toma 1
[0, 0, 1, 1, 1, 0, 0, 0, 0, 0], # toma 2
[0, 1, 0, 0, 1, 0, 1, 0, 0, 0], # toma 3
[1, 1, 0, 0, 0, 0, 1, 1, 0, 0], # toma 4
[0, 1, 0, 1, 0, 0, 0, 1, 0, 0], # toma 5
[1, 1, 0, 1, 1, 0, 0, 0, 0, 0], # toma 6
[1, 1, 0, 1, 1, 0, 0, 0, 0, 0], # toma 7
[1, 1, 0, 0, 0, 1, 0, 0, 0, 0], # toma 8
[1, 1, 0, 1, 0, 0, 0, 0, 0, 0], # toma 9
[1, 1, 0, 0, 0, 1, 0, 0, 1, 0], # toma 10
[1, 1, 1, 0, 1, 0, 0, 1, 0, 0], # toma 11
[1, 1, 1, 1, 0, 1, 0, 0, 0, 0], # toma 12
[1, 0, 0, 1, 1, 0, 0, 0, 0, 0], # toma 13
[1, 0, 1, 0, 0, 1, 0, 0, 0, 0], # toma 14
[1, 1, 0, 0, 0, 0, 1, 0, 0, 0], # toma 15
[0, 0, 0, 1, 0, 0, 0, 0, 0, 1], # toma 16
[1, 0, 1, 0, 0, 0, 0, 0, 0, 0], # toma 17
[0, 0, 1, 0, 0, 1, 0, 0, 0, 0], # toma 18
[1, 0, 1, 0, 0, 0, 0, 0, 0, 0], # toma 19
[1, 0, 1, 1, 1, 0, 0, 0, 0, 0], # toma 20
[0, 0, 0, 0, 0, 1, 0, 1, 0, 0], # toma 21
[1, 1, 1, 1, 0, 0, 0, 0, 0, 0], # toma 22
[1, 0, 1, 0, 0, 0, 0, 0, 0, 0], # toma 23
[0, 0, 1, 0, 0, 1, 0, 0, 0, 0], # toma 24
[1, 1, 0, 1, 0, 0, 0, 0, 0, 1], # toma 25
[1, 0, 1, 0, 1, 0, 0, 0, 1, 0], # toma 26
[0, 0, 0, 1, 1, 0, 0, 0, 0, 0], # toma 27
[1, 0, 0, 1, 0, 0, 0, 0, 0, 0], # toma 28
[1, 0, 0, 0, 1, 1, 0, 0, 0, 0], # toma 29
[1, 0, 0, 1, 0, 0, 0, 0, 0, 0], # toma 30
]

N_TOMAS = len(MATRIZ) # 30
N_ACTORES = len(MATRIZ[0]) # 10
MAX_TOMAS_DIA = 6 # restricción del enunciado

# --- Modelo: cada toma es una MÁSCARA DE BITS de N_ACTORES bits -----
# El bit a de la máscara vale 1 si el actor a participa en esa toma.
def fila_a_mascara(fila):
    m = 0
    for a, v in enumerate(fila):
        if v:
            m |= 1 << a
    return m

MASKS = [fila_a_mascara(fila) for fila in MATRIZ]

print(f"{N_TOMAS} tomas x {N_ACTORES} actores, máximo {MAX_TOMAS_DIA} tomas/día")
print(f"Toma 1 -> matriz {MATRIZ[0]}")
print(f"Toma 1 -> máscara {MASKS[0]:0{N_ACTORES}b} (binario) = {MASKS[0]} (entero)")

# Comprobación de integridad frente a los totales de la hoja original.
por_actor = [sum(fila[a] for fila in MATRIZ) for a in range(N_ACTORES)]
print(f"\nParticipaciones por actor: {por_actor}")
assert por_actor == [22, 14, 13, 15, 11, 8, 3, 4, 2, 2], "la matriz no

```

```
coincide"
print(f"Total de participaciones: {sum(por_actor)}")
```

30 tomas x 10 actores, máximo 6 tomas/día

Toma 1 -> matriz [1, 1, 1, 1, 1, 0, 0, 0, 0, 0]

Toma 1 -> máscara 0000011111 (binario) = 31 (entero)

Participaciones por actor: [22, 14, 13, 15, 11, 8, 3, 4, 2, 2]

Total de participaciones: 94

0.1 Contador de operaciones simples.

La herramienta de análisis de la asignatura es el **conteo de operaciones simples**, no el tiempo de reloj: los segundos dependen de la máquina y no son verificables. Por eso todos los algoritmos de este notebook incrementan un contador global **OPS** en sus operaciones elementales, y las comparaciones entre algoritmos se hacen siempre sobre ese contador.

```
In [2]: OPS = 0                                     # contador global de operaciones simples

def reset_ops():
    """Pone el contador a cero antes de medir un algoritmo."""
    global OPS
    OPS = 0

def get_ops():
    """Devuelve las operaciones simples acumuladas."""
    return OPS

def popcount(x):
    """Número de bits a 1: cuántos actores distintos hay en la
    máscara."""
    global OPS
    OPS += 1
    return bin(x).count("1")
```

1. (*) ¿Cuántas posibilidades hay sin tener en cuenta las restricciones?.

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones?.

Respuesta

Sin restricciones. Una solución es un reparto de las 30 tomas en grupos (días). Si no imponemos ni el número de días ni el tamaño de cada día, el número de soluciones es el número de formas de **partir un conjunto de n elementos**, es decir el **número de Bell** B_n , que cumple la recurrencia

$$B_n = \sum_{k=0}^{n-1} \binom{n-1}{k} B_k, \quad B_0 = 1$$

Para $n = 30$: $B_{30} = 846\,749\,014\,511\,809\,332\,450\,147 \approx 8,47 \times 10^{23}$.

Matiz importante: los días **no están etiquetados**. Grabar el grupo $\{1, 2, 3\}$ el lunes y $\{4, 5, 6\}$ el martes es la misma solución que al revés, porque el coste no depende de qué día concreto sea. Si los días se consideraran distinguibles, el recuento sería $30^{30} \approx 2,06 \times 10^{44}$, que sobrecuenta masivamente. Trabajar con particiones (y no con asignaciones etiquetadas) es la primera decisión de modelado que reduce el espacio de búsqueda, en unos **20 órdenes de magnitud**.

Con la restricción de 6 tomas por día. Si exigimos que ningún bloque supere las 6 tomas, pero dejamos libre el número de días, el recuento sigue la recurrencia (el elemento de menor índice elige a sus $j - 1$ compañeros de día):

$$P(m) = \sum_{j=1}^6 \binom{m-1}{j-1} P(m-j), \quad P(0) = 1$$

Para $m = 30$: $P(30) = 726\,391\,948\,970\,868\,949\,621\,309 \approx 7,26 \times 10^{23}$.

Con el número de días fijado a 5. Como se justifica en el apartado 2, el óptimo se alcanza usando exactamente $k = \lceil 30/6 \rceil = 5$ días de 6 tomas. El número de particiones de n elementos en k bloques de tamaño exactamente 6 es

$$\frac{n!}{(6!)^k k!}$$

El $(6!)^k$ elimina el orden **dentro** de cada día (grabar las tomas de un día en otro orden no cambia el coste) y el $k!$ elimina el orden **entre** días. Para $n = 30$, $k = 5$:

$$\frac{30!}{(6!)^5 \cdot 5!} = 11\,423\,951\,396\,577\,720 \approx 1,14 \times 10^{16}$$

Este último es el **espacio de búsqueda real** del problema. Aun habiéndolo reducido desde 10^{43} hasta 10^{16} , sigue siendo inabordable por enumeración exhaustiva: es la justificación cuantitativa de por qué hace falta un algoritmo con poda.

```
In [3]: from math import comb, factorial

# --- Sin restricciones: número de Bell -----
def bell(n):
    """B(n) = número de particiones de un conjunto de n elementos."""
    B = [1]
    for i in range(1, n + 1):
        B.append(sum(comb(i - 1, k) * B[k] for k in range(i)))
    return B[n]

# --- Con la restricción de tamaño de día, número de días libre -----
```

```

def particiones_bloques_acotados(n, tam_max):
    """Particiones de n elementos en bloques de tamaño <= tam_max."""
    P = [1] + [0] * n
    for m in range(1, n + 1):
        # el elemento de menor índice forma día con j-1 de los m-1
        # restantes
        P[m] = sum(comb(m - 1, j - 1) * P[m - j]
                   for j in range(1, min(tam_max, m) + 1))
    return P[n]

# --- Con el número de días fijado -----
def particiones_dias_completos(n, tam_dia):
    """Particiones de n elementos en n/tam_dia bloques de tamaño
    exacto."""
    k = n // tam_dia
    return factorial(n) // (factorial(tam_dia) ** k * factorial(k))

sin_restric    = bell(N_TOMAS)
etiquetados    = N_TOMAS ** N_TOMAS
con_tam        = particiones_bloques_acotados(N_TOMAS, MAX_TOMAS_DIA)
con_todo       = particiones_dias_completos(N_TOMAS, MAX_TOMAS_DIA)

print(f"Días etiquetados, sin restricciones: {etiquetados:.3e}
      ({etiquetados:,})")
print(f"Particiones sin restricciones B(30) : {sin_restric:.3e}
      ({sin_restric:,})")
print(f"Particiones con bloques <= 6          : {con_tam:.3e}
      ({con_tam:,})")
print(f"Particiones en 5 días de 6 tomas      : {con_todo:.3e}
      ({con_todo:,})")
print(f"\nReducción por el modelado: {etiquetados / con_todo:.3e} veces
      menos casos")

```

```

Días etiquetados, sin restricciones : 2.059e+44
(205,891,132,094,649,000,000,000,000,000,000,000,000,000,000,000)
Particiones sin restricciones B(30) : 8.467e+23
(846,749,014,511,809,332,450,147)
Particiones con bloques <= 6          : 7.264e+23
(726,391,948,970,868,949,621,309)
Particiones en 5 días de 6 tomas      : 1.142e+16 (11,423,951,396,577,720)

```

Reducción por el modelado: 1.802e+28 veces menos casos

2. Modelo para el espacio de soluciones.

(*) ¿Cuál es la estructura de datos que mejor se adapta al problema? Argumentalo.

Respuesta

Estructura elegida: una lista de máscaras de bits (enteros), una por toma.

La toma i se representa con un entero `MASKS[i]` de 10 bits, donde el bit a vale 1 si el actor a participa en ella. Una solución es una lista de días, y cada día es un conjunto

de índices de tomas.

El proceso de decisión (y por qué se cambió de estructura). La primera representación natural es la matriz de listas del enunciado, o un `set` de actores por toma. Con esa estructura, el cálculo del coste de un día exige unir conjuntos y contar elementos: `len(set().union(*[actores[t] for t in dia]))`, que recorre todos los actores de todas las tomas del día. Al perfilar el algoritmo se ve que **la operación crítica es precisamente esa unión**, porque se ejecuta en cada nodo del árbol de búsqueda: es el cuello de botella absoluto.

Al pasar a máscaras de bits, esa operación se convierte en:

Operación	Con <code>set</code>	Con máscara de bits
Actores de un día	<code>s1 \ s2 \ ...</code> sobre conjuntos	<code>m1 \ m2 \ ...</code> — un <code>OR</code> entero
Coste del día	<code>len(union)</code>	<code>popcount(m)</code>
Añadir una toma a un día	copia/actualiza el conjunto	<code>m \ MASKS[t]</code>
Coste incremental de añadir	recalcular la unión entera	<code>popcount(m \ MASKS[t]) - popcount(m)</code>
Memoria por día	objeto <code>set</code> con punteros	un solo entero

La ventaja decisiva no es solo la constante: es que el **coste incremental** de añadir una toma a un día se calcula en $O(1)$ operaciones sobre enteros, sin recorrer el día. Eso es lo que hace viable la ramificación y poda, donde se añaden y quitan tomas millones de veces.

Además, como $A = 10 \leq 64$, cada máscara cabe en una palabra de máquina, y en Python los enteros pequeños hacen el `OR` en una sola operación.

Estructura de la solución. Un día se guarda como la tupla `(mascara_acumulada, num_tomas)`, de modo que las dos comprobaciones que hace el algoritmo (coste y capacidad) son inmediatas y no requieren recorrer el día.

Forma canónica. Para no visitar la misma partición varias veces cambiando el orden de los días, las tomas se recorren en un orden fijo y cada toma solo puede ir a un día ya abierto o abrir *el siguiente* día. Esto elimina el factor $k!$ del recuento del apartado 1 directamente en la estructura del árbol de búsqueda.

```
In [4]: # Ejemplo de por qué las máscaras son cómodas: coste de un día
        # cualquiera.
        dia_ejemplo = [0, 5, 6] # tomas 1, 6 y 7 (índices 0-based)

        m = 0
        for t in dia_ejemplo:
            m |= MASKS[t] # unión de actores: un OR entero
```

```

print("Tomas del día      :", [t + 1 for t in dia_ejemplo])
for t in dia_ejemplo:
    print(f"  toma {t+1:2d} -> {MASKS[t]:0{N_ACTORES}b}")
print(f"  unión -> {m:0{N_ACTORES}b}")
print("Actores que acuden  :", [a + 1 for a in range(N_ACTORES) if m >>
a & 1])
print("Coste del día      :", popcount(m), "días-actor")

# Coste incremental de añadir la toma 11 (índice 10) a ese día: O(1).
delta = popcount(m | MASKS[10]) - popcount(m)
print(f"\nAñadir la toma 11 costaría {delta} actor(es) nuevo(s)")

```

```

Tomas del día      : [1, 6, 7]
  toma  1 -> 0000011111
  toma  6 -> 0000011011
  toma  7 -> 0000011011
  unión -> 0000011111
Actores que acuden : [1, 2, 3, 4, 5]
Coste del día      : 5 días-actor

Añadir la toma 11 costaría 1 actor(es) nuevo(s)

```

3. Según el modelo para el espacio de soluciones.

(*) ¿Cuál es la función objetivo?

(*) ¿Es un problema de maximización o minimización?

Respuesta

Función objetivo. Sea $S = \{D_1, D_2, \dots, D_k\}$ una partición de las tomas en días, y sea M_t la máscara de actores de la toma t . El número de actores que deben desplazarse el día D_j es el cardinal de la unión de los actores de sus tomas. Como todos cobran lo mismo por día, el gasto total es proporcional a:

$$f(S) = \sum_{j=1}^k \left| \bigcup_{t \in D_j} M_t \right| = \sum_{j=1}^k \text{popcount} \left(\bigvee_{t \in D_j} M_t \right)$$

sujeto a:

- $\bigcup_j D_j = \{1, \dots, 30\}$ y $D_i \cap D_j = \emptyset$ — cada toma se graba exactamente una vez;
- $|D_j| \leq 6$ para todo j — no más de 6 tomas por día.

La unidad de f son **días-actor**: el número de veces que algún actor tiene que desplazarse al estudio. El gasto en euros es $f(S)$ multiplicado por la tarifa diaria, que es constante y por tanto irrelevante para la optimización.

Es un problema de MINIMIZACIÓN: se busca $S^* = \arg \min_S f(S)$.

¿Por qué el óptimo usa exactamente 5 días? La función objetivo es *subaditiva* respecto a la fusión de días: para dos días cualesquiera, $|A \cup B| \leq |A| + |B|$. Fusionar dos días nunca empeora el coste; lo único que lo impide es la restricción de capacidad. Como $\lceil 30/6 \rceil = 5$, ninguna solución puede usar menos de 5 días. En el apartado 6 se comprueba además **experimentalmente** que permitir 6 o 7 días no mejora el óptimo, lo que respalda fijar $k = 5$ en el modelo.

Cotas del valor objetivo. El actor a participa en n_a tomas y un día admite como mucho 6, luego debe acudir al menos $\lceil n_a/6 \rceil$ días. Esto da una **cota inferior válida para cualquier solución**:

$$f(S) \geq \sum_{a=1}^A \left\lceil \frac{n_a}{6} \right\rceil$$

que para estos datos vale **21**. La cota superior trivial es grabar cada día las 6 tomas que toquen: cualquier solución factible sirve.

```
In [5]: def coste(particion, masks=None):
        """Función objetivo: suma de actores distintos que acuden cada día."""
        global OPS
        masks = MASKS if masks is None else masks
        total = 0
        for dia in particion:
            m = 0
            for t in dia:
                m |= masks[t]                # unión de actores del día
                OPS += 1
            total += popcount(m)             # nº de actores distintos
        return total

def cota_inferior_global(masks=None, n_actores=N_ACTORES,
tam_dia=MAX_TOMAS_DIA):
    """Cota inferior del óptimo, válida para CUALQUIER solución factible.

    El actor a participa en n_a tomas y un día admite como mucho
    `tam_dia`,
    luego acude al menos ceil(n_a / tam_dia) días.
    """
    from math import ceil
    masks = MASKS if masks is None else masks
    return sum(ceil(sum(1 for m in masks if m >> a & 1) / tam_dia)
                for a in range(n_actores))

# Ejemplo: una planificación ingenua, las tomas en el orden en que
vienen.
trivial = [tuple(range(i, min(i + MAX_TOMAS_DIA, N_TOMAS))
                  for i in range(0, N_TOMAS, MAX_TOMAS_DIA))]
print("Planificación trivial (tomas en orden):")
for j, dia in enumerate(trivial, 1):
    print(f" día {j}: tomas {[t+1 for t in dia]}")
print(f"\nCoste de la planificación trivial: {coste(trivial)} días-actor")
```



```
print(f"Cota inferior del óptimo : {cota_inferior_global()} días-actor")
```

Planificación trivial (tomas en orden):

```
día 1: tomas [1, 2, 3, 4, 5, 6]
día 2: tomas [7, 8, 9, 10, 11, 12]
día 3: tomas [13, 14, 15, 16, 17, 18]
día 4: tomas [19, 20, 21, 22, 23, 24]
día 5: tomas [25, 26, 27, 28, 29, 30]
```

Coste de la planificación trivial: 38 días-actor

Cota inferior del óptimo : 21 días-actor

4. Diseña un algoritmo para resolver el problema por fuerza bruta.

Respuesta

El algoritmo por fuerza bruta **enumera todas las particiones** de las n tomas en $k = n/6$ días de 6 tomas, evalúa la función objetivo en cada una y se queda con la mejor. No usa ninguna información del problema para descartar candidatos.

La única sofisticación es generar cada partición **una sola vez**. Un generador ingenuo que elija "6 tomas para el día 1, luego 6 para el día 2, ..." produce la misma partición $k!$ veces, una por cada orden de los días. Para evitarlo se usa la **forma canónica**: en cada paso se fija la **toma de menor índice todavía sin asignar** y solo se eligen sus 5 compañeros de día. Así el día que contiene la toma 1 se construye siempre primero, el que contiene la menor toma restante después, etc., y cada partición se emite exactamente una vez.

```
FUERZA_BRUTA(tomas):
    mejor ← ∞
    para cada partición P de tomas en días de 6 (forma
    canónica):
        c ← coste(P)
        si c < mejor:
            mejor ← c; solución ← P
    devolver solución, mejor

PARTICIONES(tomas):
    si tomas está vacío: emitir la partición vacía
    primera ← tomas[0]
    para cada combinación C de 5 elementos de tomas[1:]:
        bloque ← {primera} ∪ C
        para cada sub-partición S de tomas \ bloque:
            emitir [bloque] + S
```

```
In [6]: from itertools import combinations

def particiones_canonicas(tomas, tam_dia=MAX_TOMAS_DIA):
    """Genera cada partición de `tomas` en días de `tam_dia` UNA sola
```

vez.

Forma canónica: la toma de menor índice sin asignar siempre encabeza el

día que se está construyendo, así que los días nunca se permutan entre sí.

```
"""
if not tomas:
    yield []
    return
primera, resto = tomas[0], tomas[1:]
for companeras in combinations(resto, tam_dia - 1):
    bloque = (primera,) + companeras
    elegidas = set(companeras)
    restantes = [t for t in resto if t not in elegidas]
    for sub in particiones_canonicas(restantes, tam_dia):
        yield [bloque] + sub

def fuerza_bruta(tomas, tam_dia=MAX_TOMAS_DIA, masks=None):
    """Explora TODAS las particiones y devuelve la de coste mínimo.

    Devuelve (particion, coste, num_particiones_evaluadas).
    """
    tomas = list(tomas)
    if len(tomas) % tam_dia != 0:
        raise ValueError("la fuerza bruta exige n múltiplo del tamaño de
día")
    mejor, mejor_coste, evaluadas = None, float("inf"), 0
    for particion in particiones_canonicas(tomas, tam_dia):
        evaluadas += 1
        c = coste(particion, masks)
        if c < mejor_coste:
            mejor, mejor_coste = particion, c
    return mejor, mejor_coste, evaluadas
```

```
In [7]: # Comprobación en instancias reducidas (las primeras n tomas del
problema).
# n = 30 es inabordable por fuerza bruta: 1,14e16 particiones.
print(f'{"n":>3} | {"particiones":>12} | {"coste óptimo":>12} |
{"operaciones":>14}')
print("-" * 52)
resultados_bruta = {}
for n in (6, 12, 18):
    reset_ops()
    _, c, evaluadas = fuerza_bruta(range(n))
    resultados_bruta[n] = (c, evaluadas, get_ops())
    print(f'{"n":>3} | {"evaluadas":>12,} | {"c":>12} | {"get_ops():>14,}')

```

n	particiones	coste óptimo	operaciones
6	1	7	7
12	462	14	6,468
18	2,858,856	19	60,035,976

5. Calcula la complejidad del algoritmo por fuerza bruta.

Respuesta

Sea n el número de tomas, A el número de actores y $k = n/6$ el número de días. El algoritmo genera todas las particiones canónicas y evalúa cada una.

Número de particiones generadas. Es exactamente el recuento del apartado 1:

$$N(n) = \frac{n!}{(6!)^{n/6} (n/6)!}$$

Coste de evaluar una partición. `coste` recorre las n tomas haciendo un `OR` por toma y un `popcount` por día: $\Theta(n + k) = \Theta(n)$ operaciones simples (los enteros son de $A \leq 64$ bits, así que `OR` y `popcount` cuestan $O(1)$).

Complejidad total:

$$T(n) \in \Theta\left(n \cdot \frac{n!}{(6!)^{n/6} (n/6)!}\right)$$

Es una complejidad **superexponencial**: crece más deprisa que c^n para cualquier constante c . Aplicando Stirling, $\log N(n) \sim \frac{5n}{6} \log n$, de modo que $N(n) = n^{\Theta(n)}$.

Lectura práctica. Cada 6 tomas añadidas el número de casos se multiplica por un factor que a su vez crece: de $n=18$ a $n=24$ se multiplica por ~ 33.650 , y de $n=24$ a $n=30$ por ~ 118.750 . Para $n = 30$ son $1,14 \times 10^{16}$ particiones; aun evaluando 10 millones por segundo, el algoritmo tardaría más de **36 años**. La fuerza bruta queda por tanto descartada como método de resolución, y solo se usa aquí como **oráculo de validación** en instancias pequeñas.

```
In [8]: # Contraste entre la fórmula teórica y el número real de particiones
generadas.
print(f"{'n':>3} | {'fórmula n!/((6!)^k k!)':>24} | {'generadas':>12} |
{'coinciden':>9}")
print("-" * 58)
for n in (6, 12, 18):
    teorico = particiones_dias_completos(n, MAX_TOMAS_DIA)
    _, evaluadas, _ = resultados_bruta[n]
    print(f"{'n':>3} | {teorico:>24,} | {evaluadas:>12,} | {str(teorico ==
evaluadas):>9}")

print("\nProyección del crecimiento (no se ejecuta, solo se cuenta):")
print(f"{'n':>3} | {'particiones':>29} | {'factor vs n-6':>14}")
print("-" * 53)
prev = None
for n in (6, 12, 18, 24, 30, 36):
    v = particiones_dias_completos(n, MAX_TOMAS_DIA)
    factor = f"x{v // prev:,}" if prev else "-"
    print(f"{'n':>3} | {v:>29,} | {factor:>14}")
    prev = v
```

```
# Estimación del tiempo a un ritmo hipotético de 10^7 particiones por
segundo.
casos = particiones_dias_completos(30, MAX_TOMAS_DIA)
print(f"\nA 10^7 particiones/s, n=30 exigiría {casos / 1e7 / 3.15e7:,.1f}
años")
```

n	fórmula $n!/((6!)^k k!)$	generadas	coinciden
6	1	1	True
12	462	462	True
18	2,858,856	2,858,856	True

Proyección del crecimiento (no se ejecuta, solo se cuenta):

n	particiones	factor vs n-6
6	1	-
12	462	x462
18	2,858,856	x6,188
24	96,197,645,544	x33,649
30	11,423,951,396,577,720	x118,755
36	3,708,580,189,773,818,399,040	x324,632

A 10^7 particiones/s, n=30 exigiría 36.3 años

6. (*) Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta por qué crees que mejora el algoritmo por fuerza bruta.

Respuesta

El algoritmo elegido es **ramificación y poda** (*branch & bound*), que es una técnica exacta: devuelve el óptimo **y lo demuestra**, a diferencia de una heurística.

Ramificación

Las tomas se recorren en un orden fijo. La toma i -ésima puede:

1. **unirse a un día ya abierto** que tenga hueco (menos de 6 tomas), o
2. **abrir el siguiente día**, si aún no se han abierto los 5.

Al permitir abrir únicamente *el siguiente* día (y no un día con un índice arbitrario) se impone la forma canónica: cada partición se alcanza por un único camino del árbol. Esto ya elimina el factor $k! = 120$ respecto a una enumeración con días etiquetados.

Orden de las tomas. Se procesan primero las tomas con más actores. Son las decisiones más caras, y tomarlas cerca de la raíz hace que el coste acumulado crezca rápido, lo que dispara la poda mucho antes.

Poda: la cota inferior

En cada nodo se calcula una **cota inferior admisible** del coste que aún queda por pagar. Para cada actor a :

- sean r_a las tomas pendientes en las que participa;
- sea c_a el número de **huecos libres en días ya abiertos donde a ya está pagado** (ir a esos días le sale gratis);
- las $r_a - c_a$ tomas restantes obligan a que a acuda a días adicionales, y cada día absorbe como mucho 6 de ellas.

$$\text{cota}(i) = \sum_{a=1}^A \left\lceil \frac{\max(0, r_a - c_a)}{6} \right\rceil$$

La cota es **admisible** (nunca sobreestima el coste pendiente), lo que garantiza que la poda no descarta el óptimo. Si $\text{coste_actual} + \text{cota}(i) \geq \text{mejor_conocido}$, la rama entera se descarta.

Cota superior inicial

Se arranca con una solución **voraz** que abre cada día con la toma que más actores involucra y lo completa con las tomas que menos actores nuevos añaden.

El voraz NO es la respuesta al problema. Se usa exclusivamente como cota superior inicial para que la poda empiece a funcionar desde el primer nodo. De hecho, más abajo se comprueba que **el voraz es bloqueante**: da 31 días-actor frente al óptimo de 27, un 14,8 % peor. Es precisamente la evidencia de que una técnica voraz aplicada directamente no resuelve este problema, y de por qué hace falta la búsqueda exacta con poda.

¿Por qué mejora a la fuerza bruta?

Por tres razones acumulativas:

1. **No enumera soluciones completas, sino prefijos.** Descartar un nodo a profundidad i elimina de golpe **todas** las particiones que empiezan por ese prefijo, que son millones.
2. **La cota inferior es informativa.** Como los actores frecuentes (el actor 1 aparece en 22 de 30 tomas) fuerzan mucho coste pendiente, la cota crece deprisa y corta ramas muy arriba del árbol.
3. **El coste es incremental.** Gracias a las máscaras de bits, añadir una toma a un día cuesta $O(1)$; la fuerza bruta reevalúa la partición entera, $\Theta(n)$, para cada candidata.

```
In [9]: from math import ceil

def voraz(masks, tam_dia=MAX_TOMAS_DIA, n_dias=None):
    """Solución constructiva rápida. SÓLO se usa como cota superior
    inicial.
```

```

    Abre cada día con la toma que más actores involucra y lo completa con
    las
    tomas que menos actores NUEVOS añaden.
    """
    global OPS
    n = len(masks)
    pendientes = set(range(n))
    particion = []
    while pendientes:
        semilla = max(pendientes, key=lambda t: bin(masks[t]).count("1"))
        dia, m = [semilla], masks[semilla]
        pendientes.remove(semilla)
        while len(dia) < tam_dia and pendientes:
            mejor_t, mejor_delta = None, None
            for t in pendientes:
                OPS += 1
                delta = popcount(m | masks[t]) - popcount(m)
                if mejor_delta is None or delta < mejor_delta:
                    mejor_t, mejor_delta = t, delta
            dia.append(mejor_t)
            m |= masks[mejor_t]
            pendientes.remove(mejor_t)
            particion.append(tuple(dia))
    return particion, coste(particion, masks)

```

```

In [10]: def branch_and_bound(masks, n_actores=N_ACTORES, tam_dia=MAX_TOMAS_DIA,
                                n_dias=None, limite_nodos=None):
    """Ramificación y poda. Devuelve el óptimo y demuestra su
    optimalidad.

    Devuelve (particion, coste, nodos_explorados, optimo_demostrado).
    Si se agota `limite_nodos`, `optimo_demostrado` es False y el coste
    devuelto es solo la mejor solución encontrada hasta ese punto.
    """
    global OPS
    n = len(masks)
    if n_dias is None:
        n_dias = ceil(n / tam_dia)

    # Las tomas con más actores primero: las decisiones caras arriba del
    árbol
    # hacen crecer el coste acumulado antes y disparan la poda.
    orden = sorted(range(n), key=lambda t: -bin(masks[t]).count("1"))
    masks_ord = [masks[t] for t in orden]

    # suf[i][a] = nº de tomas j >= i (en el orden fijado) en las que
    actúa a.
    suf = [[0] * n_actores for _ in range(n + 1)]
    for i in range(n - 1, -1, -1):
        for a in range(n_actores):
            suf[i][a] = suf[i + 1][a] + (masks_ord[i] >> a & 1)

    # Cota superior inicial: la solución voraz (solo para arrancar la
    poda).
    part_voraz, cota_sup = voraz(masks, tam_dia, n_dias)
    mejor = [part_voraz]
    mejor_coste = [cota_sup]

```

```

    grupos = [] # grupos[d] = tomas (en el orden fijado) del día
d
    nodos = [0]
    agotado = [False]

    def cota_restante(i, dias):
        """Cota inferior admisible del coste pendiente desde la toma
i."""
        global OPS
        extra = 0
        for a in range(n_actores):
            r_a = suf[i][a]
            if r_a == 0:
                continue
            # Huecos en días donde el actor a YA está pagado: son gratis.
            cap = sum(tam_dia - cnt for m, cnt in dias if m >> a & 1)
            if r_a > cap:
                extra += ceil((r_a - cap) / tam_dia)
        OPS += n_actores
        return extra

    def explorar(i, dias, coste_actual):
        global OPS
        nodos[0] += 1
        if limite_nodos is not None and nodos[0] > limite_nodos:
            agotado[0] = True
            return

        if i == n: # solución completa
            if coste_actual < mejor_coste[0]:
                mejor_coste[0] = coste_actual
                mejor[0] = [tuple(orden[j] for j in g) for g in grupos]
            return

        # PODA 1: cota inferior.
        if coste_actual + cota_restante(i, dias) >= mejor_coste[0]:
            return

        # PODA 2: capacidad. Las tomas que quedan deben caber en los
días.
        libres = sum(tam_dia - cnt for _, cnt in dias)
        libres += (n_dias - len(dias)) * tam_dia
        if n - i > libres:
            return

        m_t = masks_ord[i]

        # RAMA A: meter la toma en un día ya abierto con hueco.
        for d in range(len(dias)):
            m, cnt = dias[d]
            if cnt >= tam_dia:
                continue
            OPS += 1
            delta = popcount(m | m_t) - popcount(m) # coste
incremental O(1)
            dias[d] = (m | m_t, cnt + 1)
            grupos[d].append(i)
            explorar(i + 1, dias, coste_actual + delta)
            grupos[d].pop()

```

```

        dias[d] = (m, cnt)
        if agotado[0]:
            return

        # RAMA B: abrir el siguiente día (forma canónica: nunca un día
        # arbitrario).
        if len(dias) < n_dias:
            dias.append((m_t, 1))
            grupos.append([i])
            explorar(i + 1, dias, coste_actual + popcount(m_t))
            grupos.pop()
            dias.pop()

    explorar(0, [], 0)
    return mejor[0], mejor_coste[0], nodos[0], not agotado[0]

```

In [11]: # --- Validación cruzada: B&B debe coincidir con la fuerza bruta -----

```

print("Validación: ¿da B&B el mismo óptimo que la fuerza bruta?\n")
print(f"{'n':>3} | {'bruta':>6} {'ops':>14} | {'B&B':>5} {'ops':>10} {'nodos':>8} | "
      f"{'=':>5} | {'mejora':>10}")
print("-" * 78)
comparativa = []
for n in (6, 12, 18):
    c_b, _, ops_b = resultados_bruta[n]
    reset_ops()
    _, c_r, nodos, opt = branch_and_bound(MASKS[:n])
    ops_r = get_ops()
    comparativa.append((n, c_b, ops_b, c_r, ops_r, nodos))
    ratio = f"{ops_b / ops_r:,.1f}x" if ops_b >= ops_r else "no aplica"
    print(f"{'n':>3} | {'c_b':>6} {'ops_b':>14,} | {'c_r':>5} {'ops_r':>10,} {'nodos':>8,} | "
          f"{'str(c_b == c_r):>5} | {'ratio':>10}")

```

Validación: ¿da B&B el mismo óptimo que la fuerza bruta?

n	bruta	ops	B&B	ops	nodos	=	
6	7	7	7	62	1	True	no
12	14	6,468	14	936	59	True	
18	19	60,035,976	19	9,536	723	True	

6 aplica 6.9x 6,295.7x

In [12]: # --- Resolución de la instancia completa: 30 tomas -----

```

# ATENCIÓN: esta celda explora ~3,9 millones de nodos. No es instantánea.
import functools

reset_ops()
solucion, optimo, nodos_bb, demostrado = branch_and_bound(MASKS)
ops_bb = get_ops()

print(f"Óptimo : {optimo} días-actor")
print(f"Optimalidad demostrada: {demostrado}")
print(f"Nodos explorados : {nodos_bb:,}")

```



```

print(f"Operaciones simples : {ops_bb:,}")
print(f"Cota inferior      : {cota_inferior_global()} días-actor\n")

print("PLANIFICACIÓN ÓPTIMA")
print("-" * 62)
total = 0
for j, dia in enumerate(solucion, 1):
    m = functools.reduce(lambda x, y: x | y, (MASKS[t] for t in dia))
    actores = [a + 1 for a in range(N_ACTORES) if m >> a & 1]
    total += len(actores)
    print(f"Día {j}: tomas {sorted(t+1 for t in dia)}")
    print(f"      actores {actores} -> {len(actores)} días-actor")
print("-" * 62)
print(f"TOTAL: {total} días-actor")

# Verificaciones de que la solución es válida.
plana = sorted(t for dia in solucion for t in dia)
assert plana == list(range(N_TOMAS)), "alguna toma falta o se repite"
assert all(len(d) <= MAX_TOMAS_DIA for d in solucion), "algún día excede 6 tomas"
assert total == optimo, "el coste recalculado no coincide"
print("\nVerificado: todas las tomas exactamente una vez, ningún día supera 6 tomas.")

```

```

Óptimo          : 27 días-actor
Optimalidad demostrada: True
Nodos explorados : 3,907,791
Operaciones simples : 49,153,024
Cota inferior    : 21 días-actor

```

PLANIFICACIÓN ÓPTIMA

```

-----
Día 1: tomas [1, 6, 10, 11, 12, 26]
      actores [1, 2, 3, 4, 5, 6, 8, 9] -> 8 días-actor
Día 2: tomas [3, 4, 8, 15, 21, 29]
      actores [1, 2, 5, 6, 7, 8] -> 6 días-actor
Día 3: tomas [2, 7, 13, 20, 22, 27]
      actores [1, 2, 3, 4, 5] -> 5 días-actor
Día 4: tomas [5, 9, 16, 25, 28, 30]
      actores [1, 2, 4, 8, 10] -> 5 días-actor
Día 5: tomas [14, 17, 18, 19, 23, 24]
      actores [1, 3, 6] -> 3 días-actor
-----

```

TOTAL: 27 días-actor

Verificado: todas las tomas exactamente una vez, ningún día supera 6 tomas.

```

In [13]: # --- El voraz es bloqueante: no alcanza el óptimo -----
-----
reset_ops()
part_v, coste_v = voraz(MASKS)
ops_v = get_ops()

print(f"Solución voraz : {coste_v} días-actor ({ops_v:,} operaciones)")
print(f"Óptimo (B&B)    : {optimo} días-actor")
print(f"Desviación       : {(coste_v - optimo) / optimo * 100:.1f} % peor que el óptimo")
print("\nPor eso el voraz se usa únicamente como cota superior inicial de

```

```
la poda,")
print("y no como respuesta al problema.")
```

Solución voraz : 31 días-actor (1,160 operaciones)
 Óptimo (B&B) : 27 días-actor
 Desviación : 14.8 % peor que el óptimo

Por eso el voraz se usa únicamente como cota superior inicial de la poda, y no como respuesta al problema.

```
In [14]: # --- ¿Compensa usar más de 5 días? -----
# Comprobación experimental de que fijar k = ceil(30/6) = 5 no pierde el
# óptimo.
print(f"{'días permitidos':>16} | {'óptimo':>7} | {'demostrado':>11} |
{'nodos':>12}")
print("-" * 54)
for nd in (5, 6, 7):
    reset_ops()
    _, c, nn, ok = branch_and_bound(MASKS, n_dias=nd)
    print(f"{'<= ' + str(nd):>16} | {c:>7} | {str(ok):>11} | {nn:>12,}")
```

días permitidos	óptimo	demostrado	nodos
<= 5	27	True	3,907,791
<= 6	27	True	3,948,191
<= 7	27	True	3,948,212

7. (*) Calcula la complejidad del algoritmo.

Respuesta

Hay que distinguir con cuidado el **peor caso asintótico** del **comportamiento real**, porque no mejoran en la misma medida.

Peor caso

Si la cota inferior nunca lograra podar (por ejemplo, con una matriz en la que todos los actores participan en todas las tomas), el árbol se recorrería entero. El número de hojas es el mismo recuento del apartado 1, y en cada nodo se paga el cálculo de la cota, que recorre los A actores y los k días abiertos, $\Theta(Ak)$:

$$T_{\text{peor}}(n) \in O\left(Ak \cdot \frac{n!}{(6!)^{n/6} (n/6)!}\right) = O\left(\frac{nA}{6} \cdot \frac{n!}{(6!)^{n/6} (n/6)!}\right)$$

Es decir: en el peor caso la ramificación y poda NO mejora la complejidad asintótica de la fuerza bruta. Esto es una propiedad general de branch & bound y conviene decirlo explícitamente, en lugar de afirmar una mejora asintótica que no existe.

Qué sí mejora, y por qué

Lo que la poda reduce drásticamente es el **número de nodos efectivamente explorados**. Dos factores lo explican:

1. **La forma canónica** divide el espacio por $k! = 120$ de manera garantizada, no heurística.
2. **La cota inferior** corta subárboles completos. Su efecto no es acotable a priori — depende de los datos —, pero sí es medible, y esa medición es la evidencia que sustenta el diseño.

La cota inferior en sí se calcula en $\Theta(Ak)$ por nodo, que con $A = 10$ y $k = 5$ son unas 50 operaciones: un precio muy bajo comparado con el subárbol que evita explorar.

Medición sobre operaciones simples

La comparación se hace contando operaciones, nunca segundos. Sobre las mismas instancias, ambos algoritmos devuelven **siempre el mismo coste** (validación cruzada del apartado 6), pero con un esfuerzo radicalmente distinto. La celda siguiente resume la comparación y calcula la fracción del espacio de búsqueda que realmente visita el B&B en la instancia completa.

```
In [15]: print("COMPARACIÓN EN OPERACIONES SIMPLES (mismo resultado, distinto
esfuerzo)\n")
print(f"{'n':>3} | {'particiones':>22} | {'ops bruta':>14} | {'ops
B&B':>11} | {'mejora':>10}")
print("-" * 76)
for n_i, c_b, ops_b, c_r, ops_r, nod in comparativa:
    ratio = f"{ops_b / ops_r:,.1f}x" if ops_b >= ops_r else "no aplica"
    print(f"{'n_i':>3} | {'particiones_dias_completos(n_i, 6)':>22,} |
{'ops_b:>14,} | "
          f"{'ops_r:>11,} | {'ratio:>10}")

espacio = particiones_dias_completos(N_TOMAS, MAX_TOMAS_DIA)
print(f"{'N_TOMAS:>3} | {'espacio:>22,} | {'inviabile':>14} | {'ops_bb:>11,}
| {'-':>10}")

print("\n(en n=6 solo existe UNA partición posible, así que la
comparación")
print(" no es significativa: el B&B solo paga el coste fijo de
arrancar)")

print(f"\nEn la instancia completa el B&B explora {nodos_bb:,} nodos
frente a")
print(f"{'espacio:,} particiones del espacio de búsqueda:")
print(f"solo el {nodos_bb / espacio * 100:.3e} % del total.")
```

COMPARACIÓN EN OPERACIONES SIMPLES (mismo resultado, distinto esfuerzo)

n	particiones	ops bruta	ops B&B	mejora
6	1	7	62	no aplica
12	462	6,468	936	6.9x
18	2,858,856	60,035,976	9,536	6,295.7x
30	11,423,951,396,577,720	inviable	49,153,024	–

(en $n=6$ solo existe UNA partición posible, así que la comparación no es significativa: el B&B solo paga el coste fijo de arrancar)

En la instancia completa el B&B explora 3,907,791 nodos frente a 11,423,951,396,577,720 particiones del espacio de búsqueda: solo el $3.421e-08$ % del total.

8. Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios.

Respuesta

El generador produce matrices Actores/Tomas aleatorias con tres parámetros:

- **n_tomas** y **n_actores** : el tamaño de la instancia;
- **densidad** : la probabilidad de que un actor participe en una toma dada.

La densidad es el parámetro que gobierna la dificultad real de la instancia, y el experimento del apartado siguiente muestra que su efecto es el **contrario** al que sugiere la intuición:

- **Densidad baja** ($\approx 0,1$): cada actor participa en pocas tomas, luego $\lceil n_a/6 \rceil$ es un reflejo fiel de las veces que tendrá que acudir. La cota inferior es **informativa** y la poda cierra el árbol muy deprisa.
- **Densidad alta** ($\approx 0,8$): casi todos los actores salen en casi todas las tomas, así que el coste vale aproximadamente $A \cdot k$ **se agrupe como se agrupe**. Existen entonces cantidades enormes de soluciones con costes casi idénticos, y la cota no sabe distinguir entre ellas. El algoritmo encuentra una buena solución casi de inmediato, pero **demostrar** que es la óptima le obliga a recorrer muchísimos nodos.

Es decir: lo que hace difícil una instancia no es que sea difícil encontrar una buena solución, sino que sea difícil **descartar** todas las demás.

La instancia del enunciado tiene densidad $94/300 = 0,313$, pero se resuelve en 3,9 millones de nodos mientras que una instancia aleatoria uniforme de la misma densidad agota el presupuesto. La diferencia es que los datos reales están muy **desequilibrados** (el actor 1 aparece en 22 tomas y los actores 9 y 10 en solo 2), y ese desequilibrio es justo lo que la cota inferior por actor aprovecha.

El generador garantiza que **ninguna toma queda vacía** (una toma sin actores no tiene sentido en el problema) y admite una **semilla** para que los experimentos sean reproducibles.

```
In [16]: import random

def generar_instancia(n_tomas, n_actores, densidad=0.3, semilla=None):
    """Genera una matriz Actores/Tomas aleatoria como lista de máscaras.

    `densidad` es la probabilidad de que un actor participe en una toma.
    Se garantiza que ninguna toma queda vacía. `semilla` hace el
    experimento
    reproducible.
    """
    rnd = random.Random(semilla)
    masks = []
    for _ in range(n_tomas):
        m = 0
        for a in range(n_actores):
            if rnd.random() < densidad:
                m |= 1 << a
        if m == 0:                                # ninguna toma sin
actores
            m = 1 << rnd.randrange(n_actores)
        masks.append(m)
    return masks

# Ejemplo: una instancia del mismo tamaño que la del enunciado.
ejemplo = generar_instancia(30, 10, densidad=0.313, semilla=42)
print("Instancia aleatoria (30 tomas x 10 actores, densidad 0,313,
semilla 42):\n")
for i, m in enumerate(ejemplo[:8], 1):
    print(f"  toma {i:2d}: {m:010b} -> actores "
          f"{[a+1 for a in range(10) if m >> a & 1]}")
print("  ...")
dens_real = sum(bin(m).count('1') for m in ejemplo) / (30 * 10)
print(f"\nDensidad efectiva: {dens_real:.3f}")
```

Instancia aleatoria (30 tomas x 10 actores, densidad 0,313, semilla 42):

```
toma  1: 1010001110 -> actores [2, 3, 4, 8, 10]
toma  2: 1001001101 -> actores [1, 3, 4, 7, 10]
toma  3: 0011001000 -> actores [4, 7, 8]
toma  4: 0000000010 -> actores [2]
toma  5: 0001111111 -> actores [1, 2, 3, 4, 5, 6, 7]
toma  6: 0010100011 -> actores [1, 2, 6, 8]
toma  7: 1101100000 -> actores [6, 7, 9, 10]
toma  8: 0110000000 -> actores [8, 9]
...
```

Densidad efectiva: 0.360

9. Aplica el algoritmo al juego de datos generado.

Respuesta

Se aplica el algoritmo a instancias aleatorias variando **tamaño** y **densidad**, y se compara siempre contra la cota inferior teórica. Cuando la instancia es lo bastante pequeña, se valida además contra la fuerza bruta.

```
In [17]: # --- Experimento 1: efecto de la DENSIDAD (tamaño fijo, 30x10) -----
-----
print("EFECTO DE LA DENSIDAD (30 tomas x 10 actores)\n")
print(f"{'densidad':>9} | {'cota inf':>9} | {'mejor':>6} |
{'demostrado':>11} | "
      f"{'voraz':>6} | {'nodos':>10}")
print("-" * 68)
for dens in (0.1, 0.2, 0.313, 0.5, 0.8):
    inst = generar_instancia(30, 10, densidad=dens, semilla=7)
    lb = cota_inferior_global(inst, 10)
    reset_ops()
    _, cv = voraz(inst)
    reset_ops()
    _, c, nn, ok = branch_and_bound(inst, n_actores=10,
limite_nodos=8_000_000)
    print(f"{'dens':>9.3f} | {'lb':>9} | {'c':>6} | {'str(ok)':>11} | {'cv':>6} |
{'nn':>10,}")

print("\n'demostrado' = False significa que se agotó el límite de
8.000.000 de")
print("nodos: el valor de 'mejor' es entonces una cota superior válida,
no el")
print("óptimo probado.")
print("\nObsérvese que la dificultad CRECE con la densidad: a densidad
0,1 el")
print("árbol se cierra en 38.980 nodos, y a partir de 0,313 ya no se
cierra.")
print("A densidad alta la cota inferior está muy cerca del óptimo (45
frente a")
print("47), pero hay tantas soluciones de coste casi idéntico que
descartarlas")
print("todas resulta caro: lo difícil no es hallar el óptimo, sino
demostrarlo.")
```

EFFECTO DE LA DENSIDAD (30 tomas x 10 actores)

densidad	cota inf	mejor	demostrado	voraz	nodos
0.100	12	17	True	20	38,980
0.200	18	28	True	33	3,317,250
0.313	19	32	False	33	8,000,001
0.500	29	41	False	45	8,000,001
0.800	45	47	False	50	8,000,001

'demostrado' = False significa que se agotó el límite de 8.000.000 de nodos: el valor de 'mejor' es entonces una cota superior válida, no el óptimo probado.

Obsérvese que la dificultad CRECE con la densidad: a densidad 0,1 el árbol se cierra en 38.980 nodos, y a partir de 0,313 ya no se cierra. A densidad alta la cota inferior está muy cerca del óptimo (45 frente a 47), pero hay tantas soluciones de coste casi idéntico que descartarlas todas resulta caro: lo difícil no es hallar el óptimo, sino demostrarlo.

```
In [18]: # --- Experimento 2: efecto del TAMAÑO, con validación contra fuerza
bruta ---
print("EFFECTO DEL TAMAÑO (densidad 0,3, 10 actores)\n")
print(f"{'n':>3} | {'partic.':>12} | {'bruta':>6} | {'B&B':>5} | {'=':>5}
| "
      f"{'ops bruta':>13} | {'ops B&B':>9} | {'mejora':>9}")
print("-" * 82)
for n in (6, 12, 18):
    inst = generar_instancia(n, 10, densidad=0.3, semilla=n)
    reset_ops()
    _, c_b, _ = fuerza_bruta(range(n), masks=inst)
    ops_b = get_ops()
    reset_ops()
    _, c_r, nn, ok = branch_and_bound(inst, n_actores=10)
    ops_r = get_ops()
    ratio = f"{ops_b / ops_r:,.1f}x" if ops_b >= ops_r else "no aplica"
    print(f"{'n':>3} | {'particiones_dias_completos'(n, 6):>12,} | {'c_b':>6} |
{'c_r':>5} | "
          f"{'str(c_b == c_r):>5} | {'ops_b':>13,} | {'ops_r':>9,} |
{'ratio':>9}")
```

EFFECTO DEL TAMAÑO (densidad 0,3, 10 actores)

n	partic.	bruta	B&B	=	ops bruta	ops B&B
mejora						
6	1	10	10	True	7	62
no aplica						
12	462	15	15	True	6,468	949
6.8x						
18	2,858,856	20	20	True	60,035,976	13,635
4,403.1x						

```
In [19]: # --- Experimento 3: instancias mayores que la del enunciado -----
# Se limita el nº de nodos: si se agota, el resultado es una cota
superior
# válida (la mejor solución hallada), no necesariamente el óptimo.
print("INSTANCIAS MAYORES (densidad 0,3)\n")
```

```

print(f"{'tomas':>6} {'actores':>8} | {'cota inf':>9} | {'mejor':>6} | "
      f"{'óptimo?':>8} | {'nodos':>10}")
print("-" * 62)
for n_t, n_a in ((36, 10), (42, 12), (48, 12)):
    inst = generar_instancia(n_t, n_a, densidad=0.3, semilla=n_t)
    lb = cota_inferior_global(inst, n_a)
    reset_ops()
    _, c, nn, ok = branch_and_bound(inst, n_actores=n_a,
    limite_nodos=3_000_000)
    print(f"{'n_t':>6} {'n_a':>8} | {'lb':>9} | {'c':>6} | {'str(ok)':>8} | "
          f"{'nn':>10,}")

print("\nCuando 'óptimo?' es False el valor es la mejor solución
encontrada")
print("dentro del límite de nodos, y sigue siendo una cota superior
válida.")

```

INSTANCIAS MAYORES (densidad 0,3)

tomas	actores	cota inf	mejor	óptimo?	nodos
36	10	23	36	False	3,000,001
42	12	31	62	False	3,000,001
48	12	33	62	False	3,000,001

Cuando 'óptimo?' es False el valor es la mejor solución encontrada dentro del límite de nodos, y sigue siendo una cota superior válida.

10. Enumera las referencias que has utilizado para llevar a cabo el trabajo.

Respuesta

1. **Brassard, G. y Bratley, P. (1997).** *Fundamentos de algoritmia*. Prentice Hall. ISBN 9788489660007. — Capítulos de ramificación y poda y de análisis de algoritmos; base del esquema de cota inferior admisible empleado.
2. **Guerequeta, R. y Vallecillo, A. (2000).** *Técnicas de diseño de algoritmos*. Universidad de Málaga. <http://www.lcc.uma.es/~av/Libro/indice.html> — Tratamiento de vuelta atrás y ramificación y poda con ejemplos de problemas de partición.
3. **Lee, R. C. T., Tseng, S. S., Chang, R. C. y Tsai, Y. T. (2005).** *Introducción al diseño y análisis de algoritmos*. McGraw-Hill. ISBN 9789701061244.
4. **Duarte, A. (2008).** *Metaheurísticas*. Dykinson. — Consultado para valorar alternativas metaheurísticas, finalmente descartadas al comprobar que la búsqueda exacta cierra la instancia del enunciado.
5. **Documentación de Python:** módulo `itertools` (`combinations`) y operaciones sobre enteros usadas para el modelo de máscaras de bits. <https://docs.python.org/3/library/itertools.html>

Sobre el uso de herramientas de IA. Se ha empleado un asistente basado en LLM como herramienta de apoyo y colaboración durante todo el trabajo: revisión gramatical y de estilo, ordenación y estructura de las ideas, apoyo para interpretar y razonar el porqué de los resultados obtenidos, y revisión del código. El objetivo de ese uso ha sido obtener un resultado válido y de calidad, no sustituir el criterio propio. Todos los resultados numéricos proceden de la ejecución del código aquí incluido, y la corrección del algoritmo se ha verificado mediante validación cruzada contra la fuerza bruta y comprobaciones automáticas (`assert`) sobre la solución final.

11. Describe brevemente las líneas de cómo crees que es posible avanzar en el estudio del problema.

Respuesta

1. Cotas inferiores más fuertes. La cota usada (21) está a un 22 % del óptimo (27), y esa holgura es lo que obliga a explorar 3,9 millones de nodos. Una cota basada en **relajación lineal** del problema de *set partitioning*, o en pares de actores que coinciden con frecuencia, cerraría el árbol mucho antes. Los actores 1 y 2 coinciden en muchas tomas: explotar esas correlaciones por pares daría una cota bastante más ajustada que tratar a cada actor por separado.

2. Programación dinámica sobre subconjuntos. Para $n \leq 25$ es viable un $O(3^n)$ sobre subconjuntos de tomas, que da el óptimo sin depender de la calidad de la poda. A partir de ahí el consumo de memoria lo hace inviable, pero es una alternativa exacta con complejidad **acotada a priori**, algo que branch & bound no ofrece.

3. Escalado del tamaño. El experimento 3 muestra que a partir de ~40 tomas la búsqueda exacta deja de cerrar el óptimo dentro de un presupuesto razonable de nodos. Para tamaños industriales (cientos de tomas) haría falta una metaheurística (recocido simulado o algoritmo genético con representación por máscaras), pero entonces sería imprescindible **reportar la distancia a la cota inferior** como medida de calidad, ya que se pierde la garantía de optimalidad.

4. Variantes del problema más realistas. El modelo actual es deliberadamente simple. Extensiones naturales:

- **Tarifas distintas por actor.** El coste de un día pasa a ser $\sum_{a \in D_j} w_a$ en vez de $|D_j|$. El algoritmo apenas cambia: basta sustituir `popcount` por una suma ponderada, y la cota inferior se convierte en $\sum_a w_a \lceil n_a/6 \rceil$.
- **Disponibilidad de los actores.** Si un actor no puede acudir ciertos días, los días dejan de ser intercambiables y se pierde la forma canónica, lo que agranda mucho el espacio de búsqueda.
- **Precedencias entre tomas.** Si algunas tomas deben grabarse antes que otras, el problema gana una estructura de orden parcial.

- **Duración variable de las tomas.** Sustituir "6 tomas por día" por un límite de minutos convierte la restricción de capacidad en una **mochila**, y el problema pasa a ser un *bin packing* con costes de unión.

5. Análisis de la estructura del óptimo. Es llamativo que en la solución óptima uno de los días reúna solo 3 actores mientras otro reúne 8. Estudiar si esa concentración de "tomas baratas" en un mismo día es un patrón general sugeriría reglas de descomposición para instancias grandes.